NETWORKING KERNEL REVIEW

Byeong-hee Roh
Dept. of Software and Computer Engineering
Ajou University

변화와 융합의 중심
MMcN
멀티미디어 융합 연구실
Mobile Multimedia convergecne Network Lab.
http://mmcn.ajou.ac.kr

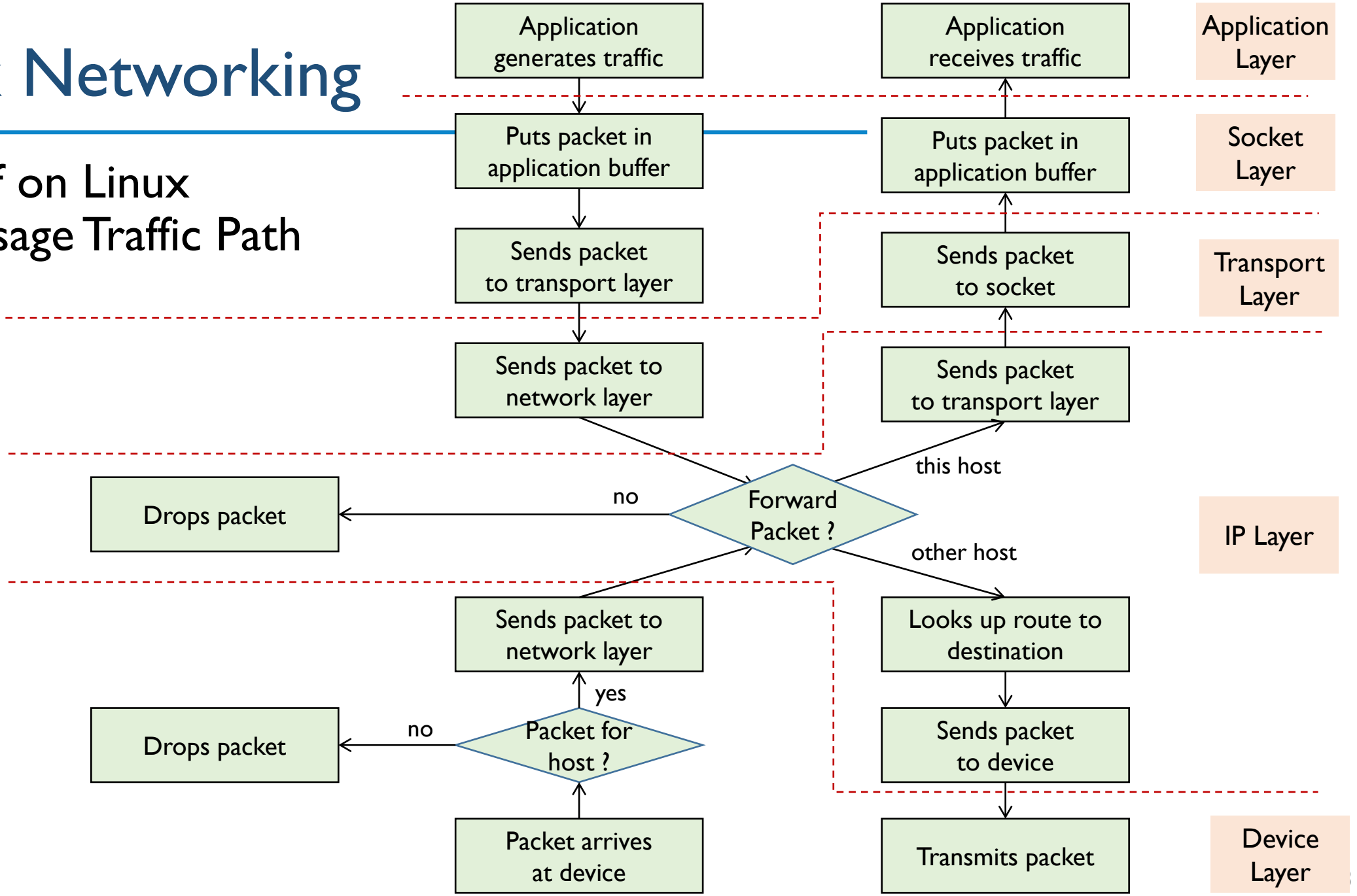
Contents

- Linux IP Networking
- Network Device Driver
- References
  - G. Herrin, Linux IP Networking: A Guide to the implementation and Modification of the Linux Protocol Stack, May 2000
  - A. Rubini, J. Corbet, Linux Device Drivers, 3rd Edition, O'Reilly Pub. Jan. 2005

LINUX IP NETWORKING

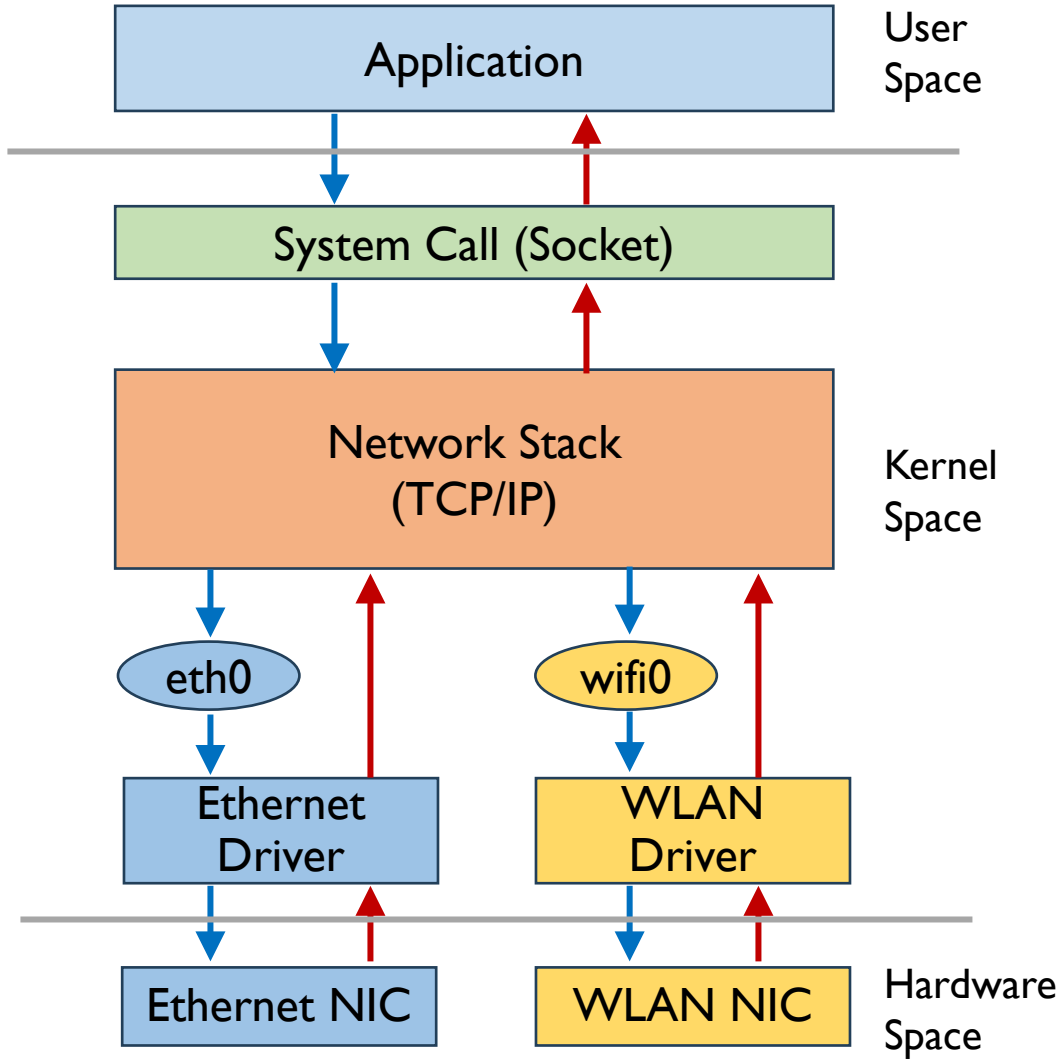
Linux Networking

- Brief on Linux Message Traffic Path



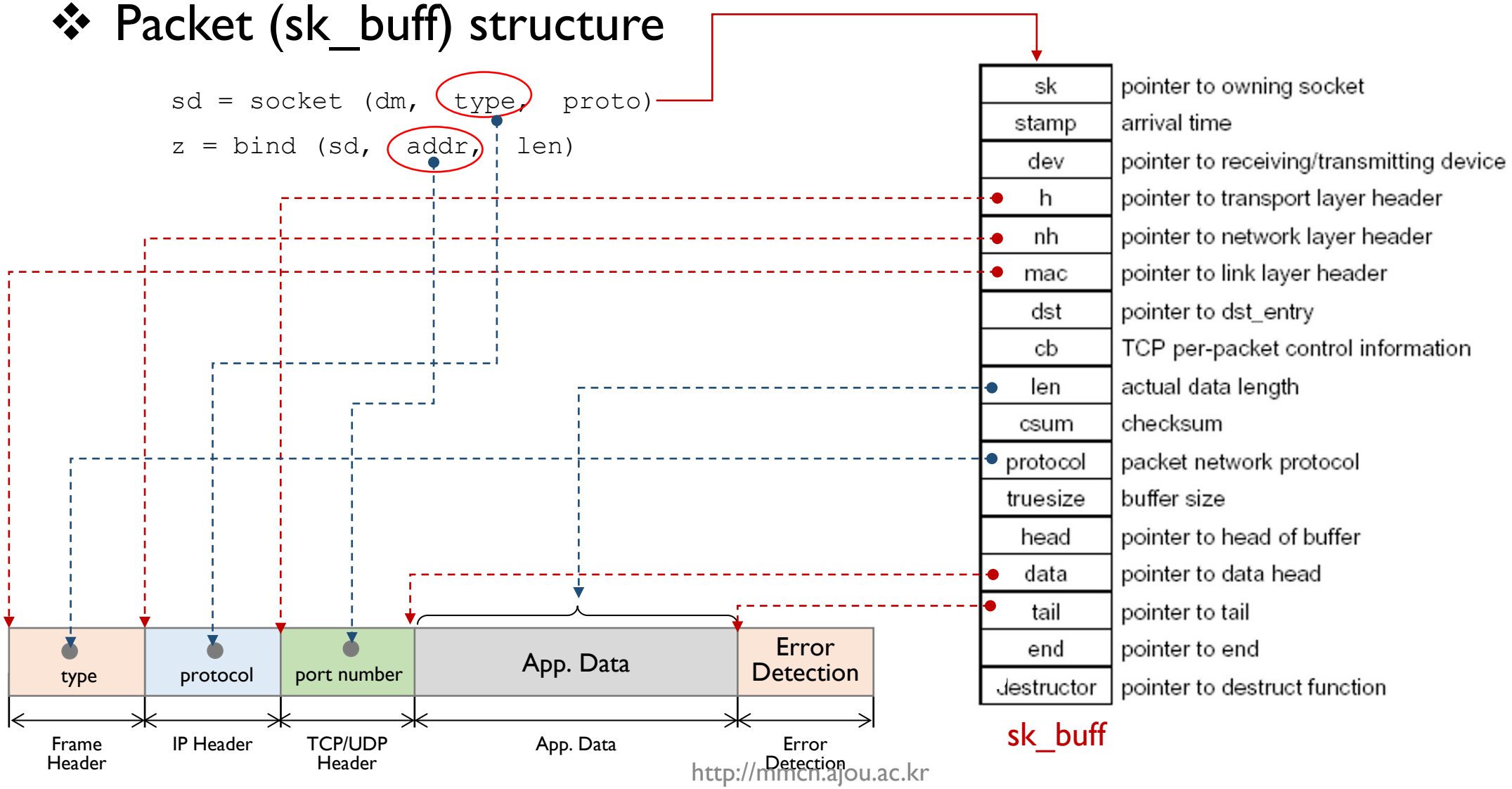
Protocol Stack

- Protocol Stack
  - Application
  - Socket
  - Transport
  - IP
  - Network Devices
- Key to maintaining the protocol stack
  - without wasting time copying parameters and payloads back and forth
  - is the use of the common packet data structure (a socket buffer)
    - it is defined in struct sk_buff



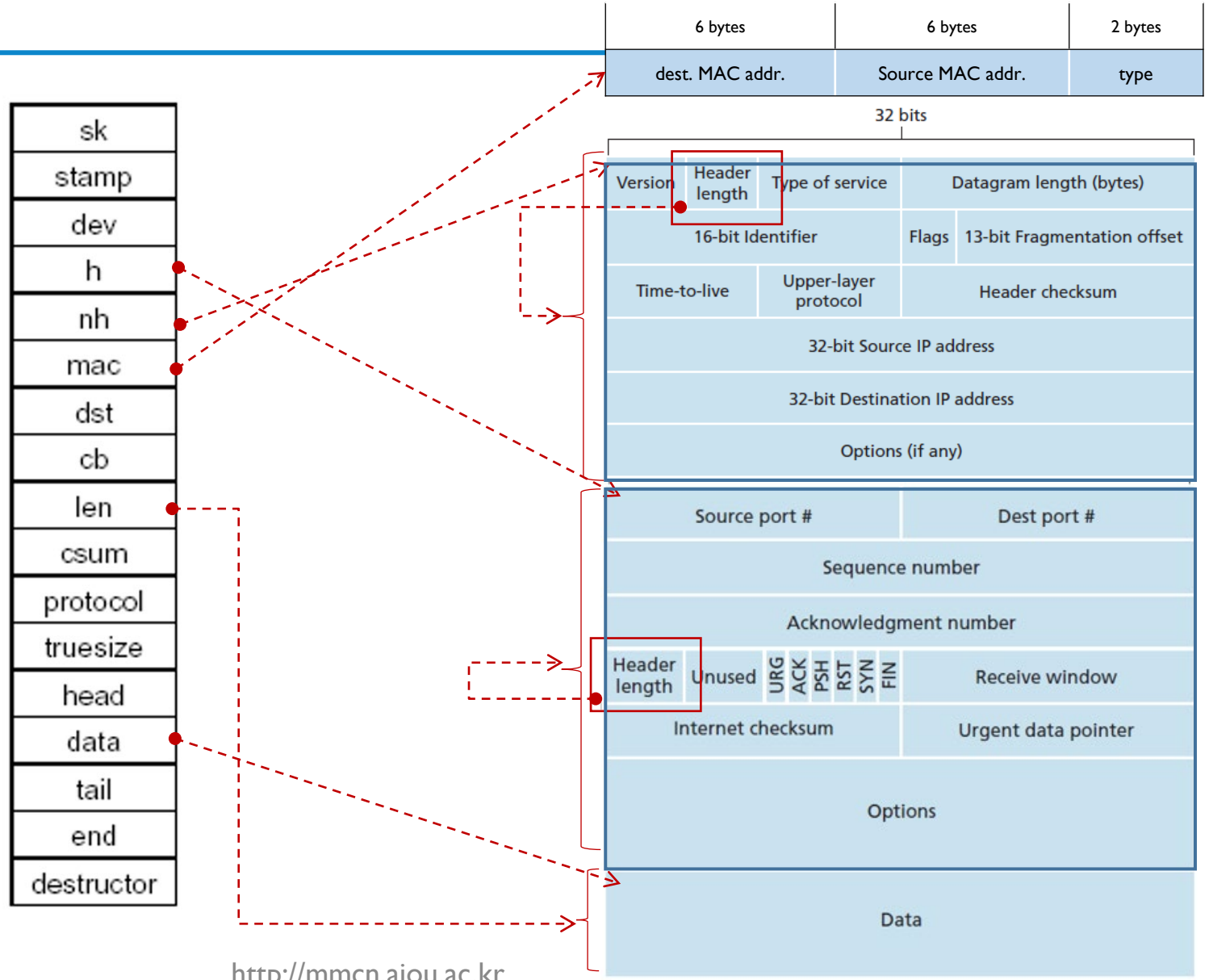
sk_buff

❖ Packet (sk_buff) structure



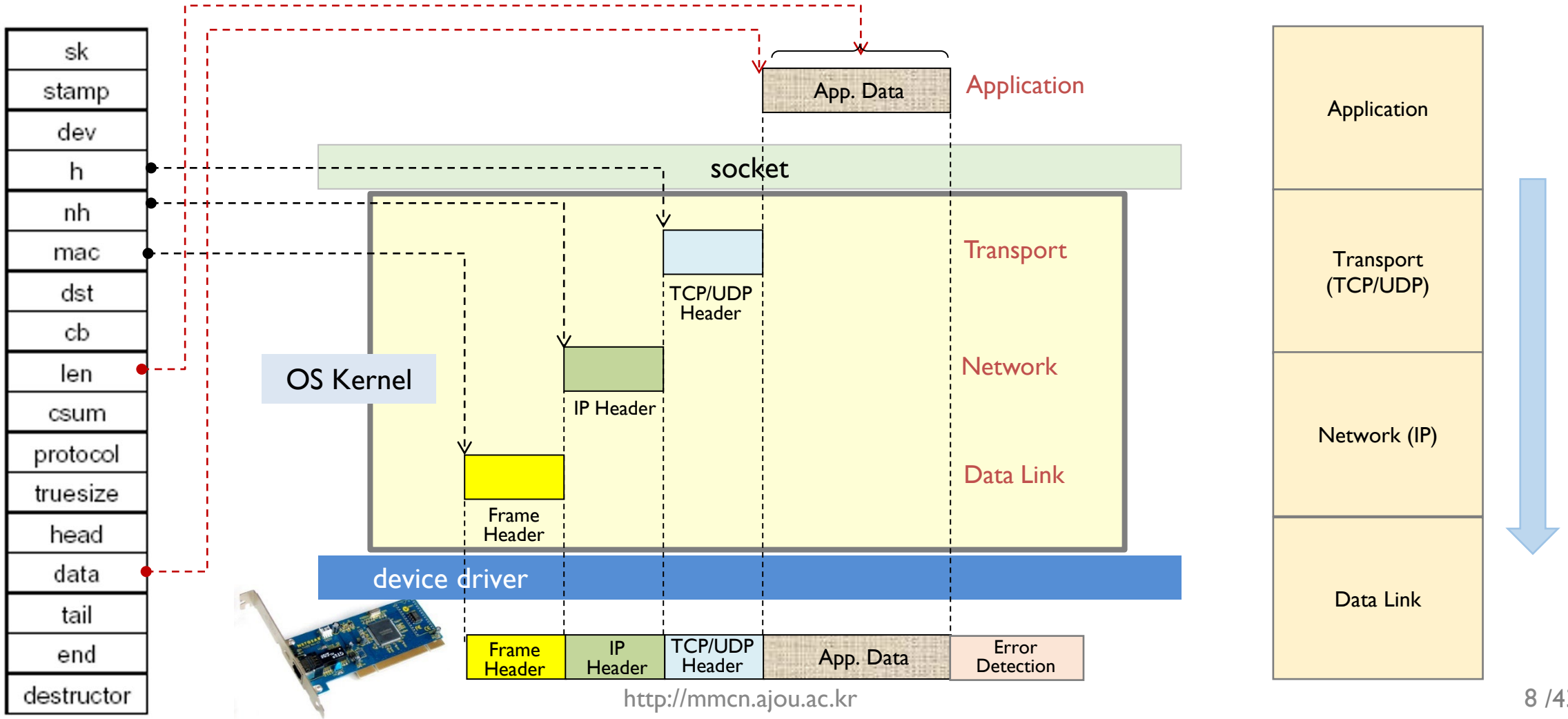
sk_buff

- Packet Headers



sk_buff

❖ Encapsulation with sk_buff



Encapsulation and Socket

- Payload Data Copy
  - it happens twice only
    - once from user to kernel space
    - once from kernel space to output medium (for an outbound packet)
  - All of the protocols (TCP/UDP, IP, Ethernet)
    - use the same socket buffer

IP Networking – Start Up

Commands for Network Initialization

- Two basic commands for network initialization
  - `ifconfig (for Linux)`
    - `ipconfig (for Windows)`
  - `route`

Functions of Network Initialization Script

e.g.) Red Hat dist. - /etc/rc.d/init.d/network

- Set environment variables to identify the host computer
- Establish whether or not the computer will use a network
- Turn on/off IP forwarding and IP fragmentation
- Establish the default router for all network traffic and
- Set the device to use to send such traffic
- Bring up any network devices using the *ifconfig* and *route*
- …

ifconfig – Network Device Info. and Configuration

- `ifconfig` (for Linux)
  - it configures interface devices for use

```
ifconfig ${DEVICE} ${IPADDR} netmask ${NMASK} broadcast ${BCAST}
```

  - it provides information about currently configured network devices

```
$ ifconfig
eth0 Link encap:Ethernet HWaddr 00:C1:4E:7D:9E:25
inet addr:172.16.1.1 Bcast:172.16.1.255 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:389016 errors:16534 dropped:0 overruns:0 frame:24522
TX packets:400845 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:100
Interrupt:11 Base address:0xcc00
```

Ref: http://www.computerhope.com/unix/uifconfi.htm

ifconfig – Network Device Info. and Configuration

- examples
  - to shut down (or activate) eth0

```
$ ifconfig eth0 down (or $ ifconfig eth0 up )
```

  - to enable (or disable) ARP on eth0

```
$ ifconfig eth0 arp (or $ ifconfig eth0 -arp )
```

  - to set the eth0 netmask

```
$ ifconfig eth0 netmask 255.255.255.0
```

  - to set the loopback maximum transfer unit

```
$ ifconfig lo mtu 2000
```

  - to set the eth1 IP address

```
$ ifconfig eth1 172.16.0.7
```

ipconfig (for windows)

❖ syntax

```
ipconfig [/allcompartments] [/? | /all |
                                /renew [adapter] | /release [adapter] |
                                /renew6 [adapter] | /release6 [adapter] |
                                /flushdns | /displaydns | /registerdns |
                                /showclassid adapter |
                                /setclassid adapter [classid] |
                                /showclassid6 adapter |
                                /setclassid6 adapter [classid] ]i
```

Routing Table and "route" command

- syntax
  - Network Stack
    - needs to determine the path (the nexthop router) to forward IP packets
  - Routing Table
    - provides the routing information for the path selection
  - "route"
    - shows the routing table
    - to see the routing table in Linux
      - $ route –n
    - to see the routing table in Windows
      - > route -PRINT

route – show/manipulate the IP routing table

❖ syntax (for Linux)

```
route [-CFvnee] [-A family |-4|-6]

route [-v] [-A family] add [-net|-host] target [netmask Nm] [gw Gw]
      [metric N] i [mss M] [window W] [irtt m] [reject] [mod] [dyn]
      [reinstate] [[dev] If]

route [-v] [-A family] del [-net|-host] target [gw Gw] [netmask Nm]
      [metric N] [[dev] If]

route [-V] [--version] [-h] [--help]
```

- for more details: https://man7.org/linux/man-pages/man8/route.8.html

route

- syntax (for Linux)
  - to add predefined routes for interface devices to the FIB
    - FIB (Forwarding Information Base): or Routing Table

```
route add -net ${NETWORK} netmask ${NMASK} dev ${DEVICE}
route add -host ${IPADDR} ${DEVICE}
```

  - to delete routes from the FIB

```
route del -net ${NETWORK} netmask ${NMASK} dev ${DEVICE}
route del -host ${IPADDR} ${DEVICE}
```

  - to display information about the routes

```
route
```

route

- examples (for Linux)
  - to add a route to a host

```
$ route add -host 127.16.1.0 eth1
```

    - packets with destination address of "127.16.1.0" should be forwarded to eth1
  - to add a network

```
$ route add -net 172.16.1.0 netmask 255.255.255.0 eth0
```

    - packets with dest. addresses of "127.16.1.0~ 127.16.1.255" should be forwarded to eth0
  - to set the default route through `jeep`

```
$ route add default gw jeep
```

    - note that a route to `jeep` must already be set up)
  - to delete entry for host 172.16.1.16

```
$ route del -host 172.16.1.16
```

route

- "route –n" command example (for Linux)
  - to show the Kernel IP routing table (or FIB)

Number of references to this route
(Not used in the Linux Kernel)

Number of lookups for the route (Depending on the use of cache hit/miss)

```
$ route
Kernel IP routing table
Destination     Gateway         Genmask         Flags Metric Ref    Use Iface
172.16.1.4      *               255.255.255.255 UH    0      0        0 eth0
172.16.1.0      *               255.255.255.0   U     0      0        0 eth0
127.0.0.0       *               255.0.0.0       U     0      0        0 lo
Default         viper.u.edu     0.0.0.0         UG    0      0        0 eth0
```

U : route is up
G : route is to a gateway
H : route is to a host(host address)

route (for Windows)

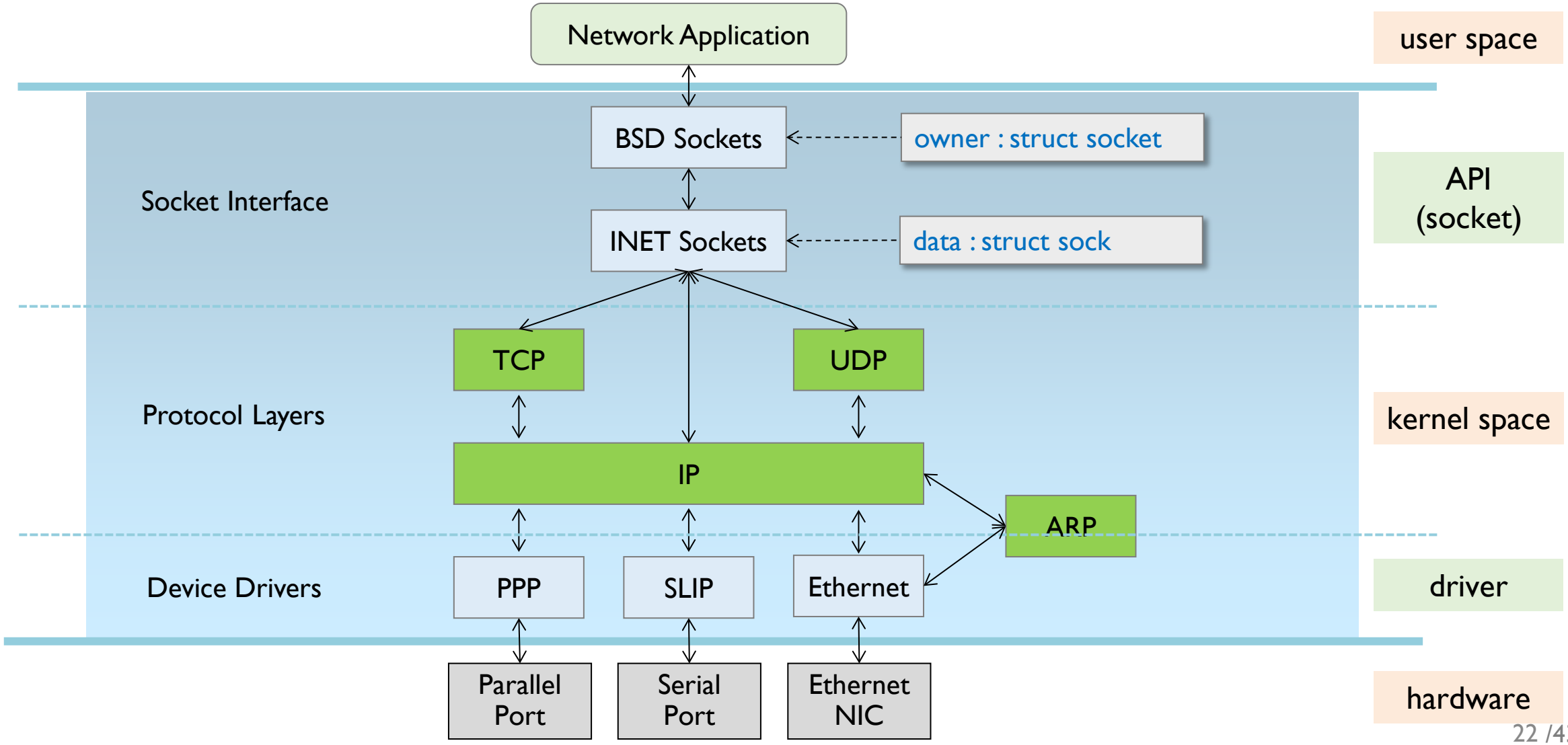
❖ syntax

```
ROUTE [-f] [-p] [-4|-6] command [destination]
      [MASK netmask]  [gateway] [METRIC metric]  [IF interface]
```

- for command
  - PRINT to show the routing table
  - ADD to add a route
  - DELETE to remove a route
  - CHANGE to modify an existing route

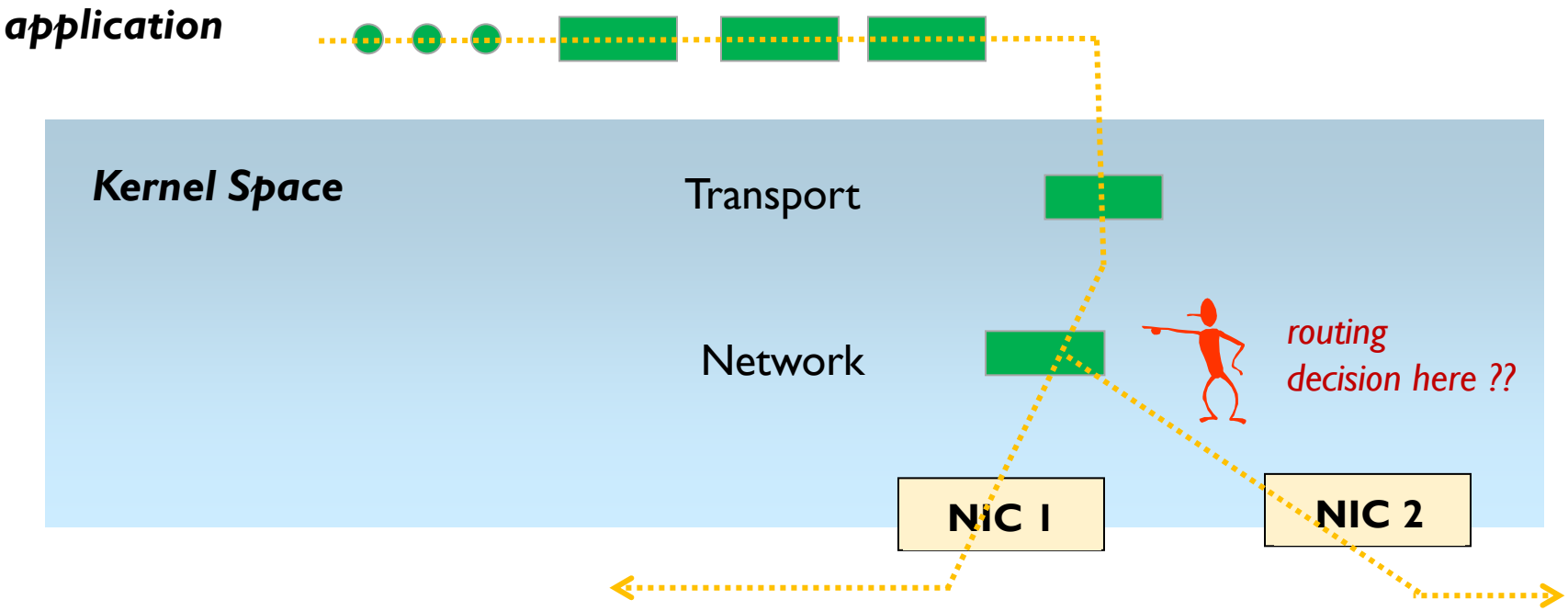
IP Networking – Connection Process

Linux Networking Hierarchy



Routing Packets

- Question
  - what happens if packets from an application are forwarded according to individual routing process ?
  - then what is advantage of connection



Routing Packets

❖ Routing Packets in Linux

- example

```
sockfd = socket(AF_INET, SOCK_STREAM, 0);

addr.sin_family = AF_INET;
addr.sin_addr   = inet_ntoa( SERVER_ADDRESS);
addr.sin_port   = htons(PORT_NUM);

connect (sockfd, &addr, sizeof addr);

write(sockfd, …);
read(scokfd,…);

close(sockfd);
```

we will look at this point
what happens in the kernel

Connection Process

- socket creation – after socket( ) call
  - Check for errors in call
  - Create (allocate memory for) socket object (e.g. sk_buff)
  - Put socket into INODE list
  - Establish pointers to protocol functions (INET)
  - Store values for socket type and protocol family
  - Set socket state to closed
  - Initialize packet queues

msg마다 개별 sk_buff사용됨.

Connection Process

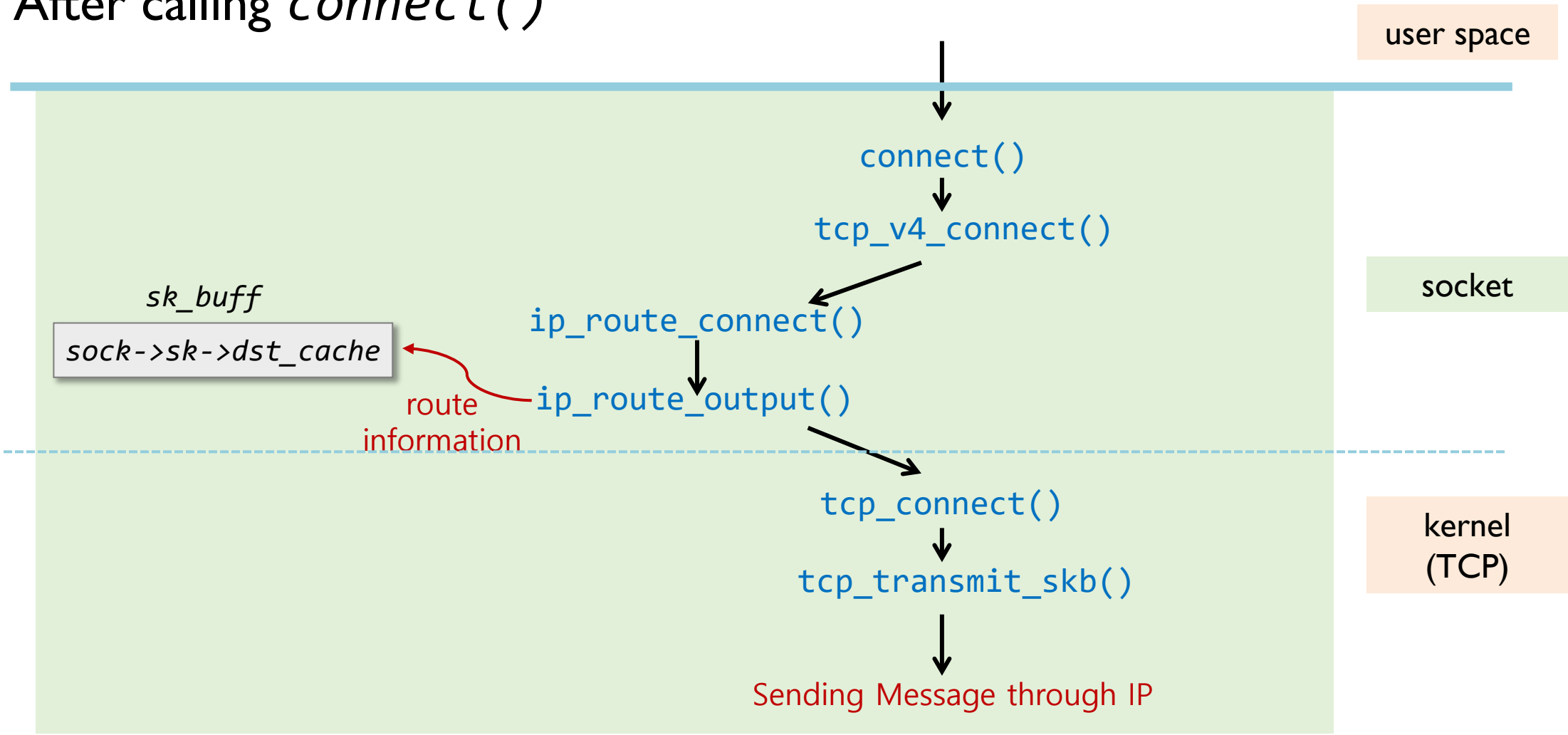
- connection setup – after connect( ) call
  - Check for errors
  - Determine route to destination:
    - Check routing table for existing entry (return that if one exists)
    - Look up destination in *FIB*
    - Build new routing table entry
    - Put entry in routing table and return it
  - Store pointer to routing entry in socket
  - Call TCP connection establishment function
    - TCP: generates a TCP SYN segment,
    - IP: encapsulates it in IP datagram, then sends
    - …
  - Set socket state to established

routing decision

TCP Connection Setup

Connection Process

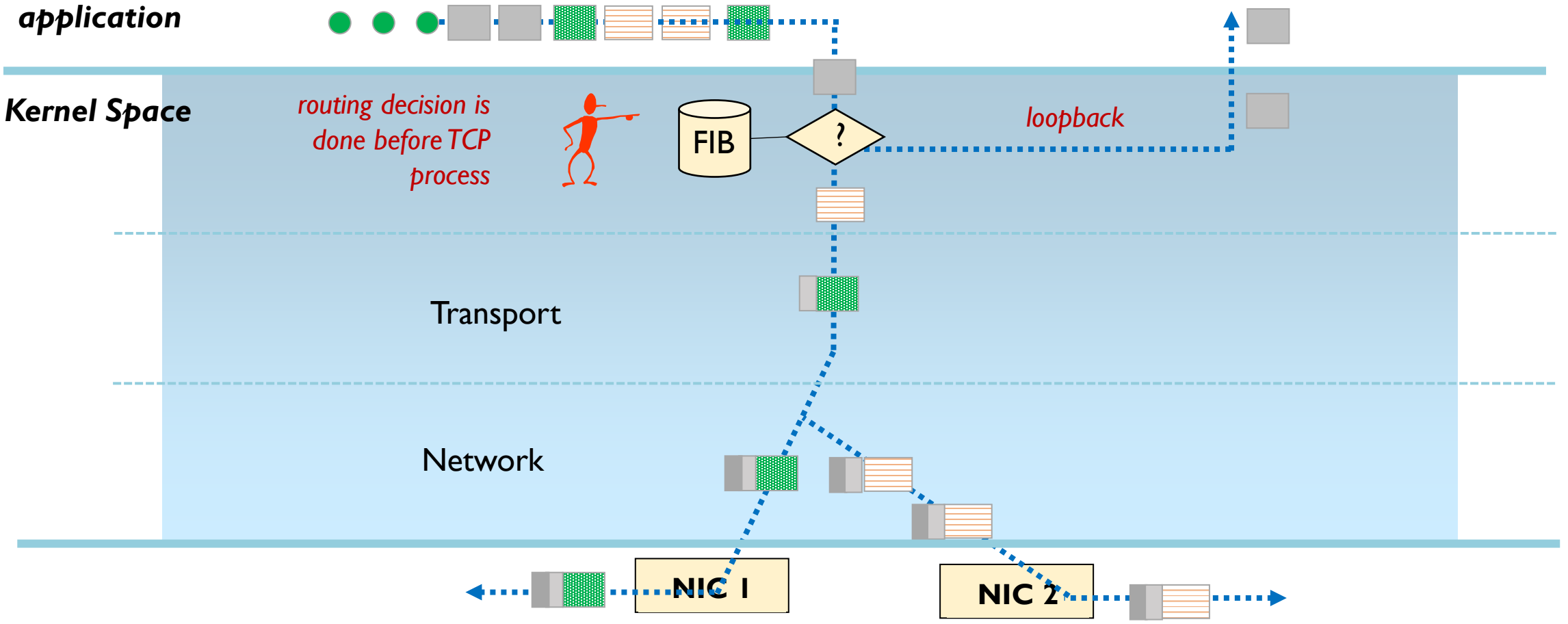
❖ After calling *connect()*



Connection Process

❖ Routing Process

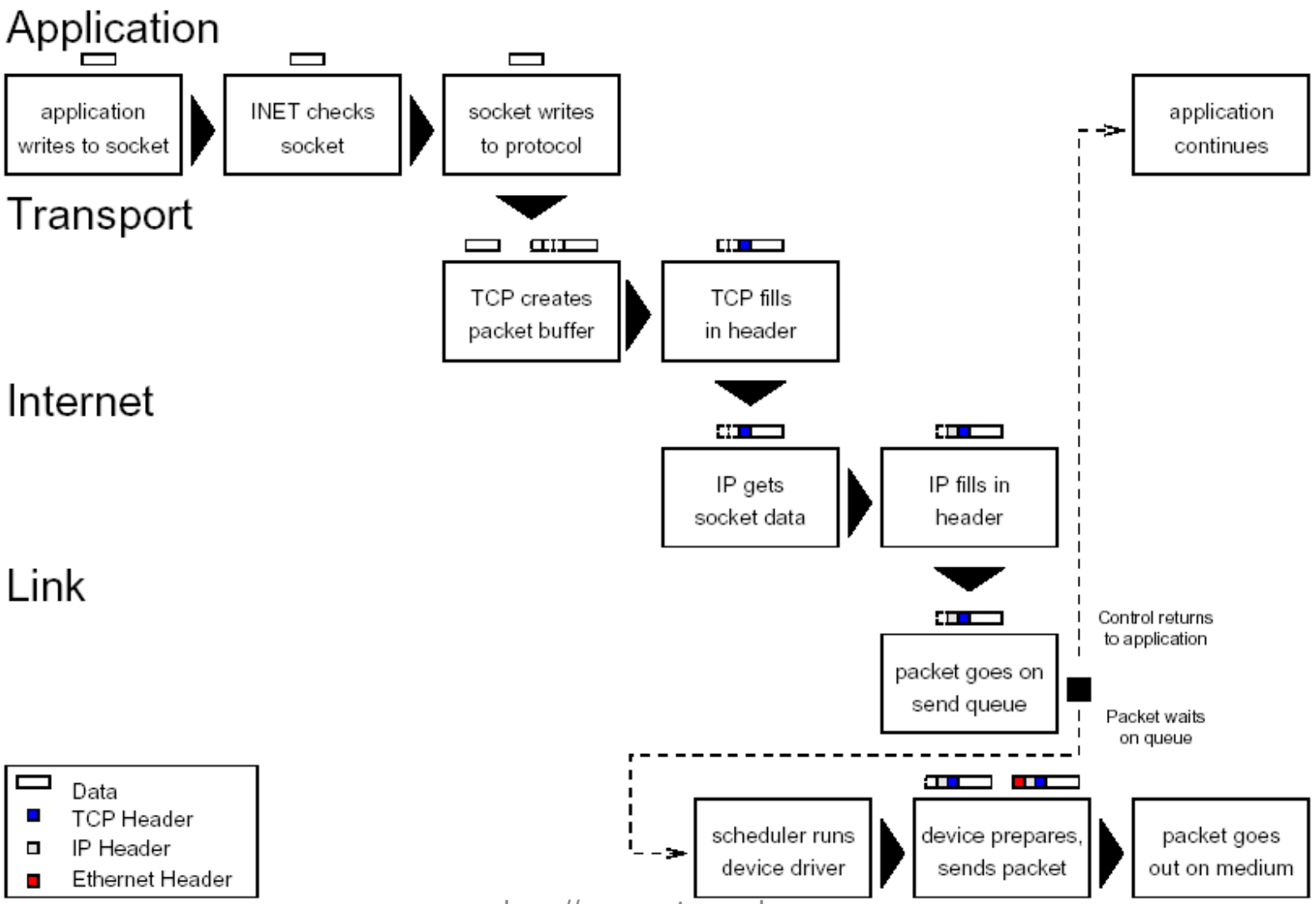
loopback은 capture안됨.(capture: datalink layer)
routing에 대한 결정은 socket layer에서 결정됨.



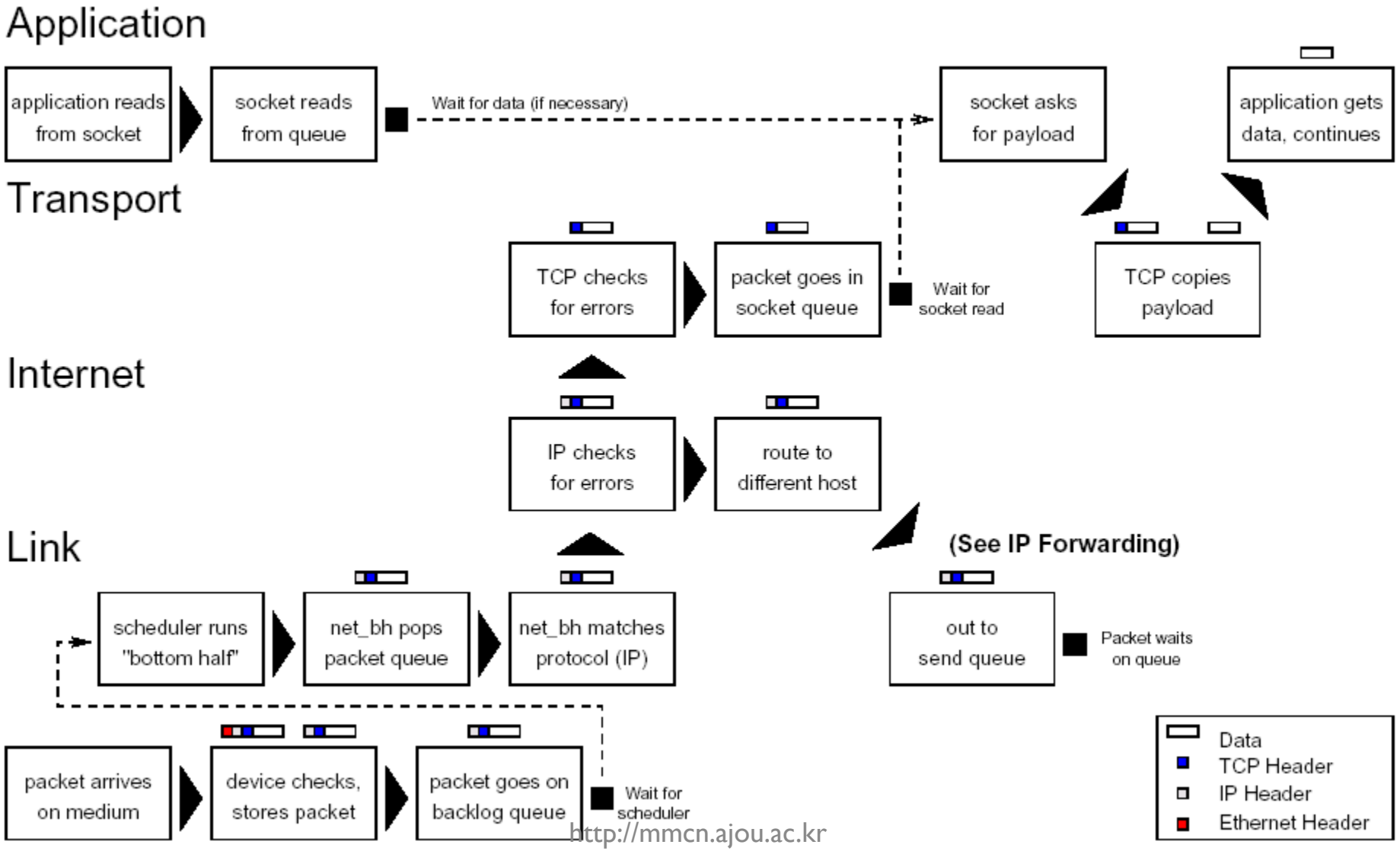
Connection Process

- socket close - after close( ) call
  - Check for errors
    - e.g. does the socket exist?
  - Change the socket state to disconnecting to prevent further use
  - Do any protocol closing actions
    - e.g., send a TCP packet with the FIN bit set
  - Free memory for socket data structures
    - TCP/UDP and INET
  - Remove socket from INODE list

Message Transmissions

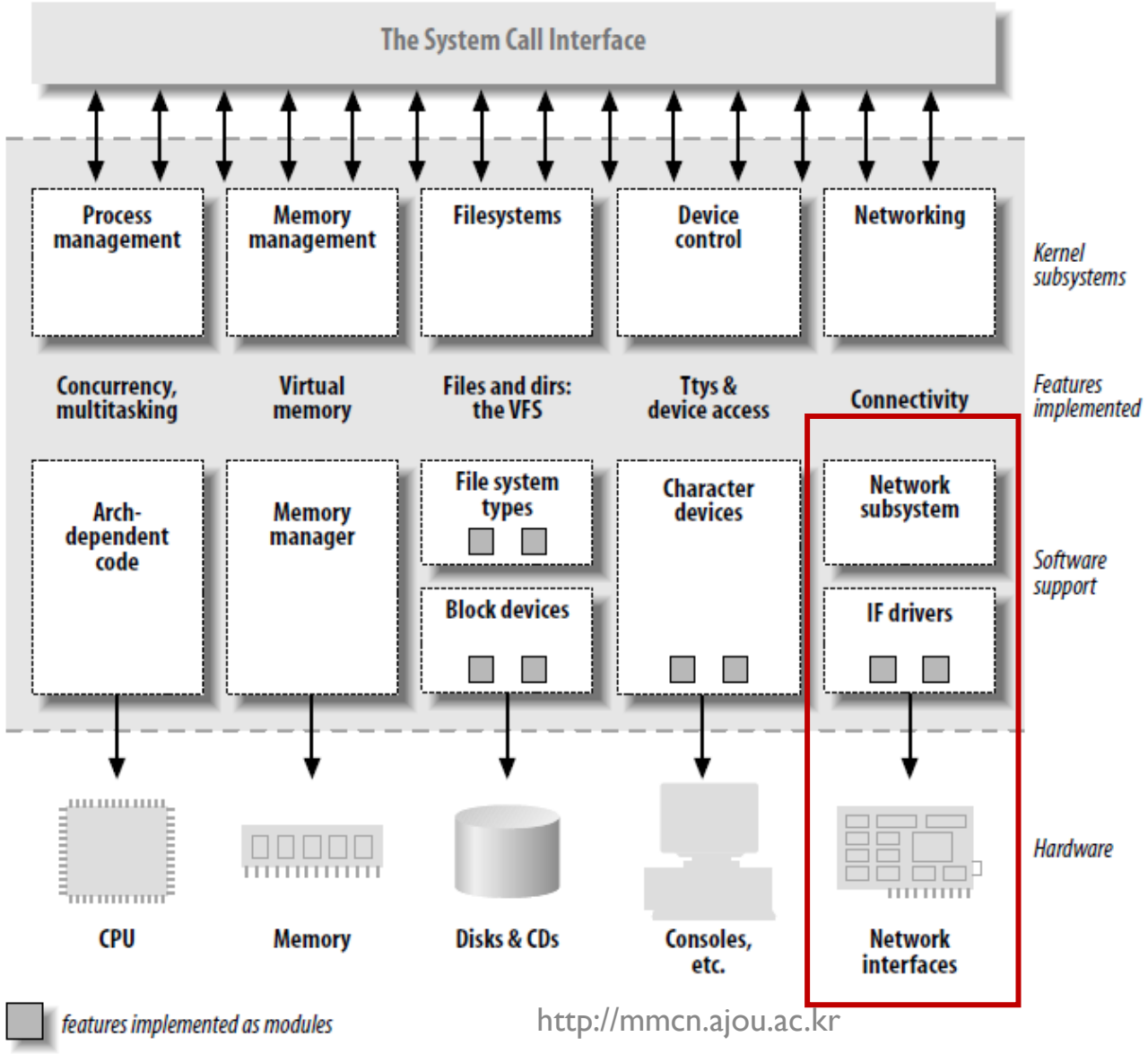


Message Receptions



IP Networking – Network Device Driver

Split View Of Linux Kernel



Split View Of Linux Kernel

- Kernel
  - the big chunk of executable code in charge of handling all requests from several concurrent processes
  - Kernel's roles
    - Process Management
      - Create, Destroy, IPC, scheduler, …
    - Memory Management
      - replace, block, cache hierarchy
    - File Systems Management
      - file abstraction, various filesystem type
    - Device Control
      - device driver for every peripheral present on a system
    - Networking
      - packet collecting, identifying, delivering, routing, address resolution, …

Device Control

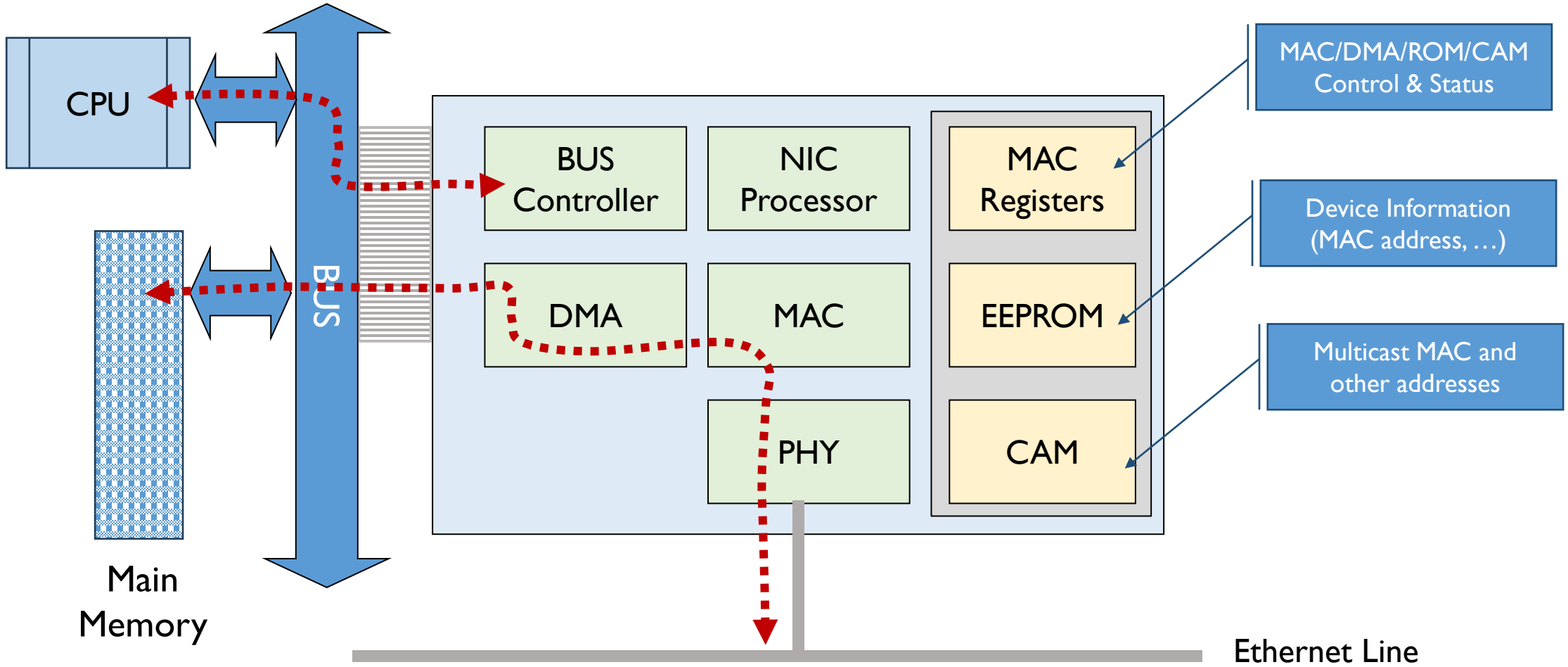
- Device
  - Almost every system operation eventually maps to a physical device
  - all device control operations are performed by code that is specific to the device
    - the code is called a device driver
- Basic Three Device Types in UNIX/LINUX
  - Character Devices
  - Block Devices
  - Network Interfaces (Devices)

Network Interfaces

- Basic Characteristics
  - it is in charge of sending and receiving data packets, driven by the network subsystem of the kernel
    - it can be carried out without any knowledge of upper layer protocols – protocol independent
  - a network device is not easily mapped to a node in the filesystem, as dev/tty I
    - it uses a unique name such as eth0
  - there exists a special data structure for the interfaces
    - struct devices is defined in /usr/src/linux/include/netdevice.h
  - the device does not see the individual streams, but only the data packets

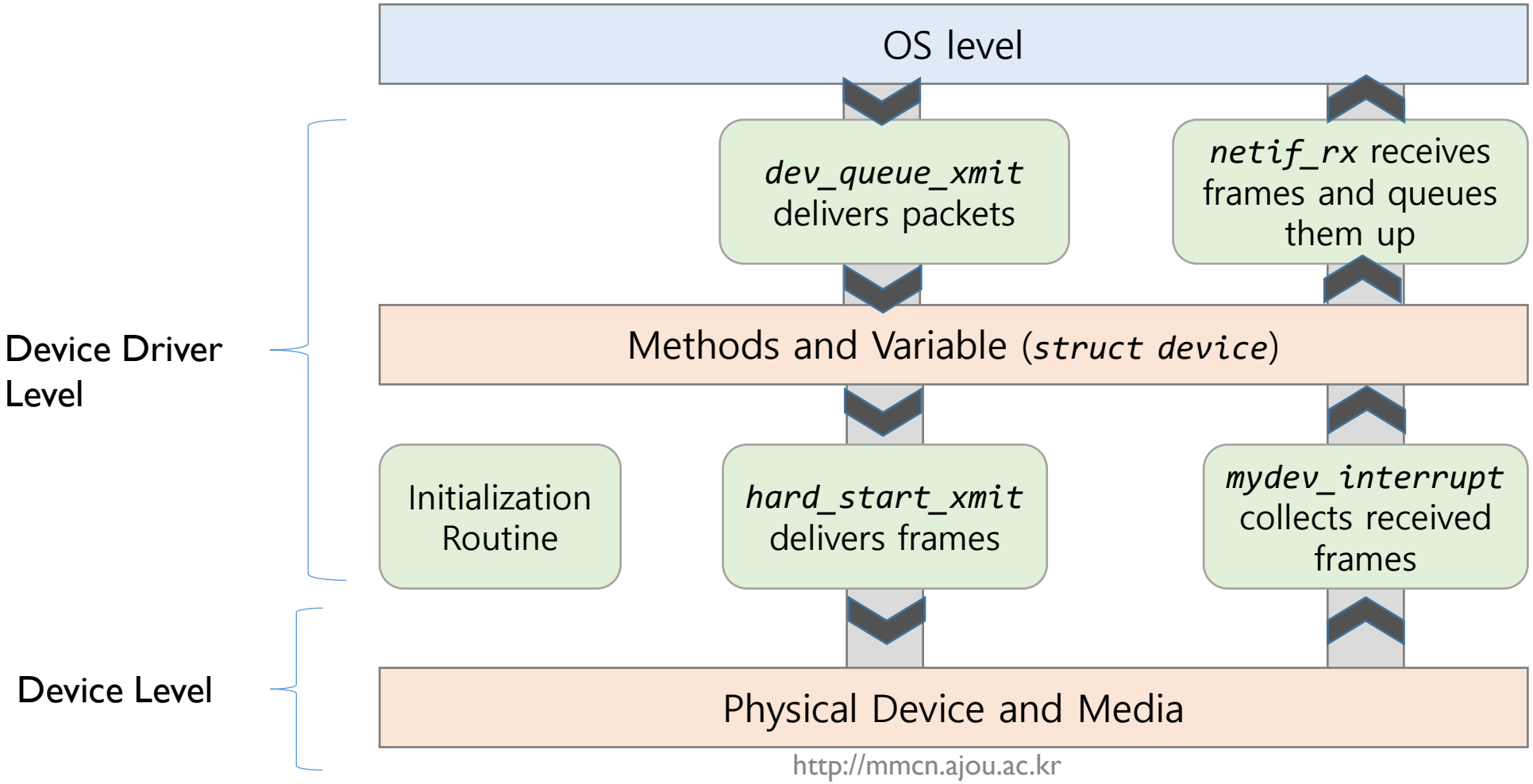
Network Interfaces

❖ Basic NIC Device Architecture



Network Device Driver

❖ Sending/Receiving Packets At Device Levels



Network Device Driver

❖ Overall Sending Process

① Application: send( ), sendto( ), connect( ), .. 등의 socket 함수를 사용하여 전송 요청

② Socket Layer: 전송을 위한 socket buffer (sk_buff)를 동적으로 할당

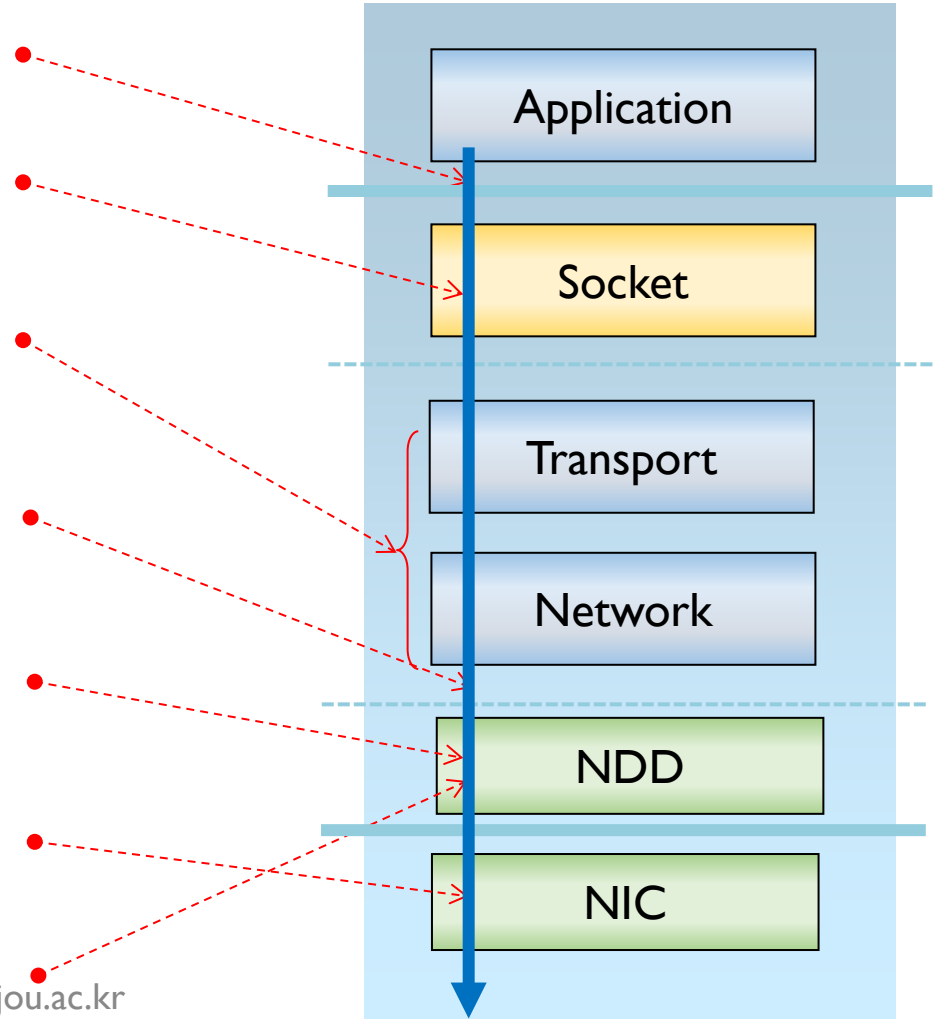
③ TCP/IP Kernel (upper part): sk_buff에 각 layer header를 작성

④ TCP/IP Kernel (lower part): dev_queue_xmit( )을 호출하여 NDD에게 실제 전송을 요청

⑤ NDD: hard_start_xmit( ) 호출하여 NIC을 통한 실제 전송 작업 수행

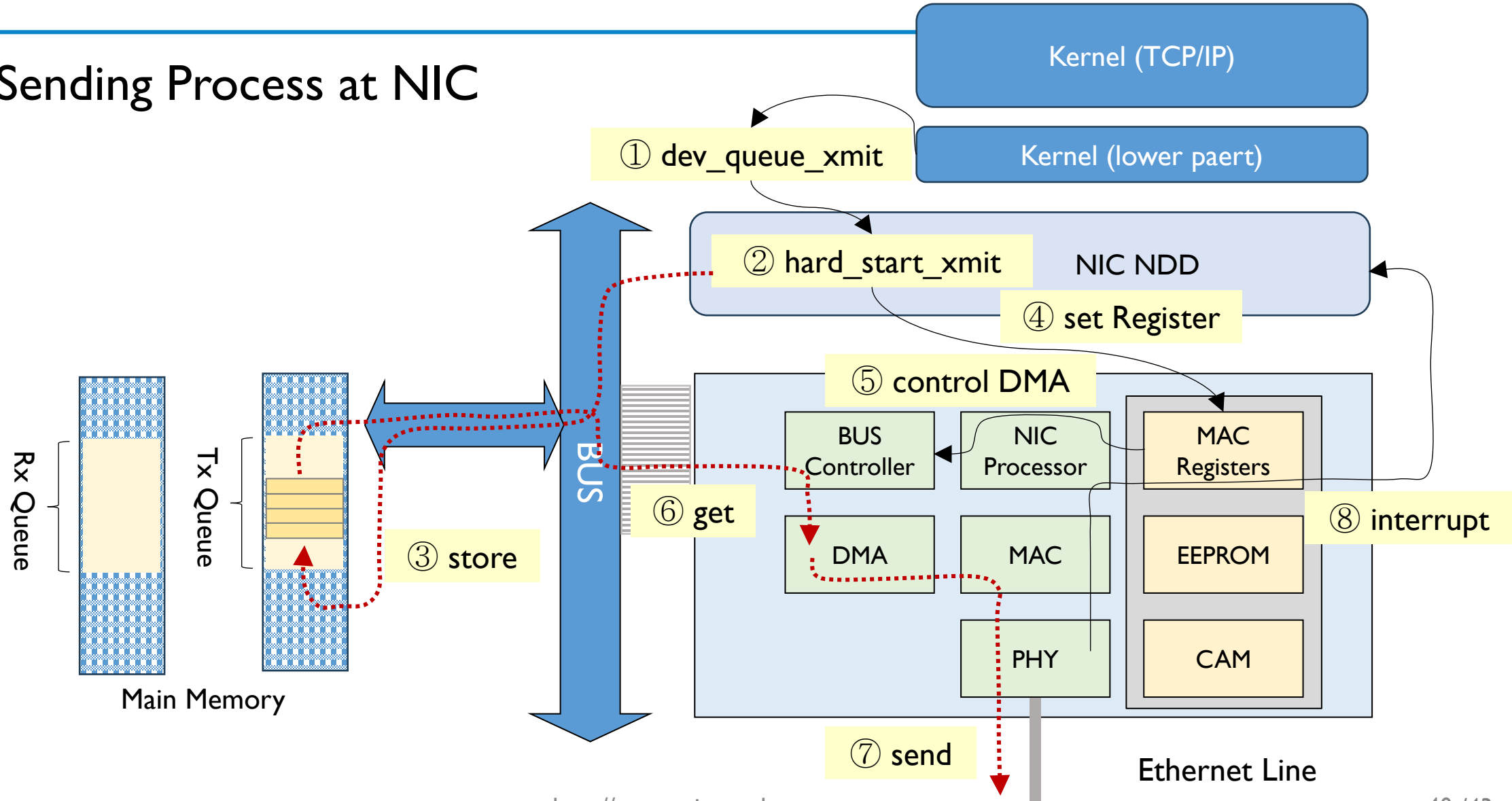
⑥ NIC: 전송후 interrupt 발생

⑦ NDD: socket buffer (sk_buff) 해지



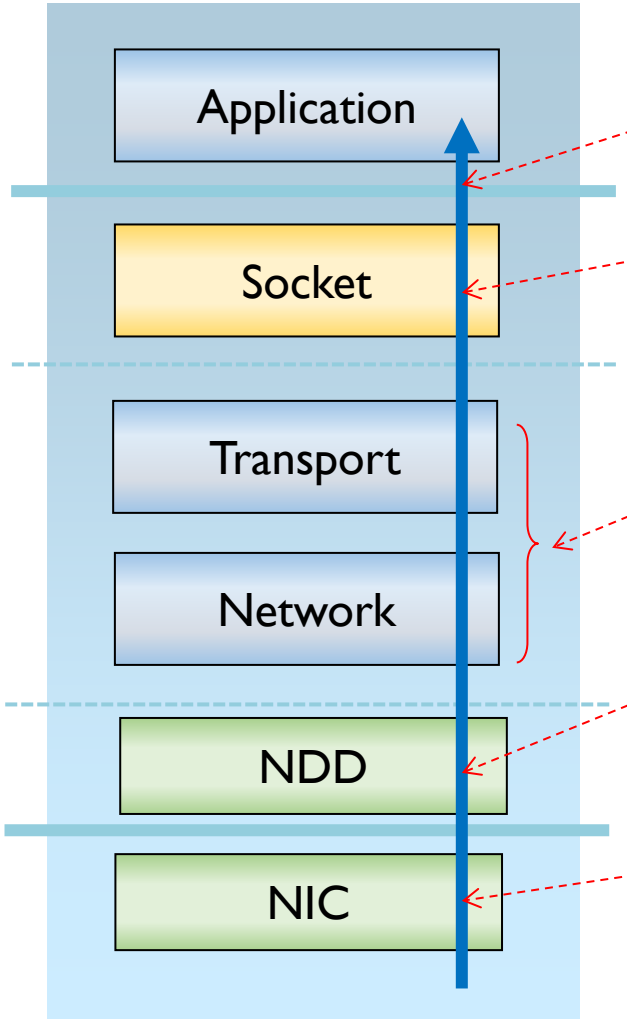
Network Device Driver

❖ Sending Process at NIC



Network Interfaces

❖ Receiving Process



⑤ Application:
recvfrom( ), recv( ), accept( )… 와 같은 receive 관련 API 함수가 return됨 :

④ Socket Layer:
응용에게 데이터를 전달(copy)하면서 socket buffer (sk_buff) 해지

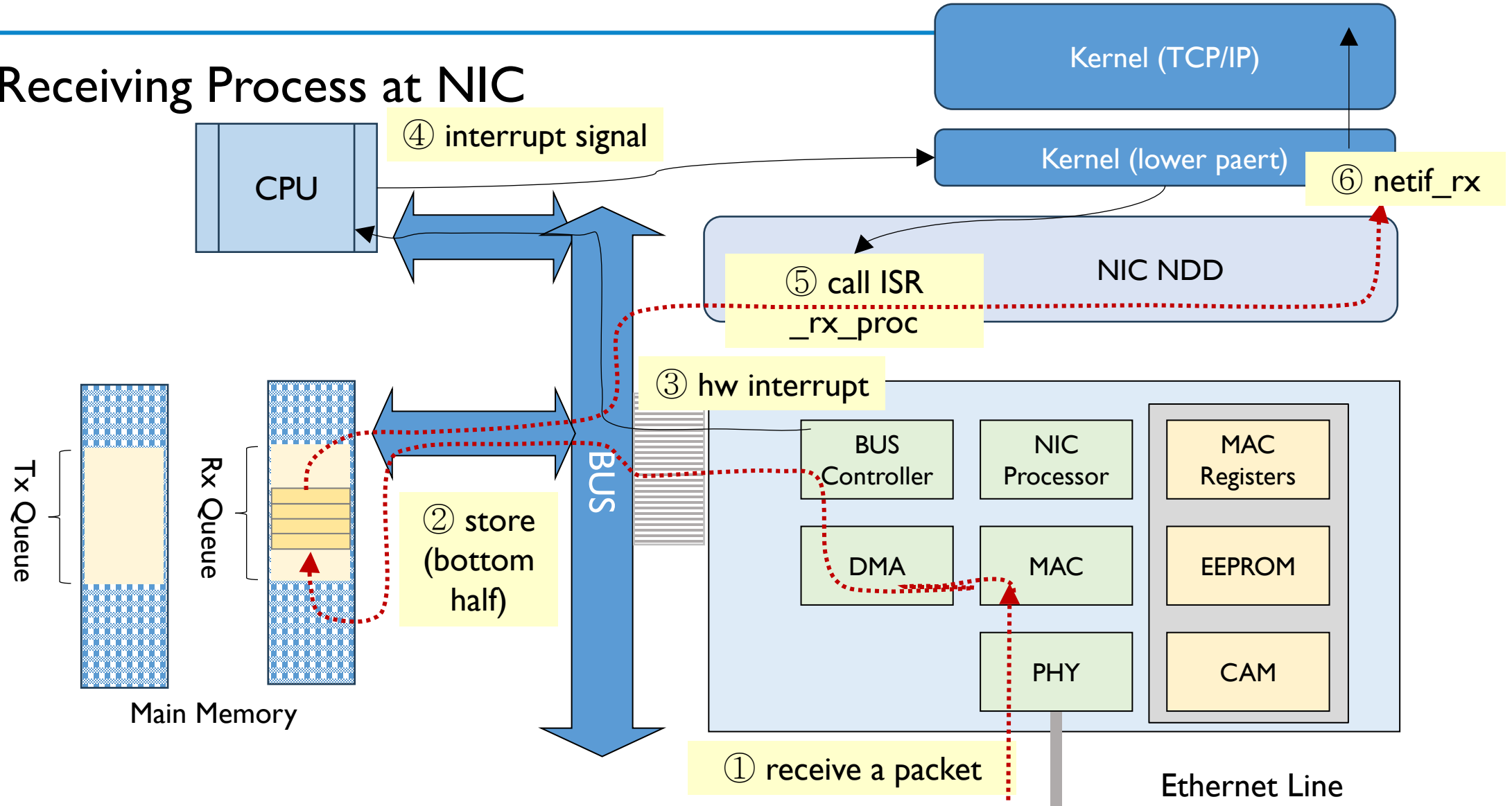
③ TCP/IP Kernel:
NDD가 할당한 socket buffer를 사용하여 처리

② NDD:
- 수신 처리를 위한 sk_buff 동적 할당
- 프레임을 미리 설정된 구조에 맞추어 메모리에 저장
- netif_rx ( )를 호출

① NIC:
수신 프레임의 dest. MAC 일치시, Interrupt를 발생하여 NDD의 관련 ISR을 호출

Network Device Driver

❖ Receiving Process at NIC



Questions ?

MR-IoT/AI Convergence

Future Internet & 5G/6G

Intelligent Networking with AI

MMcN

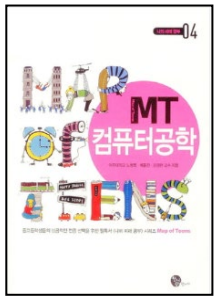
Mobile **Multimedia Convergence** Network Lab.

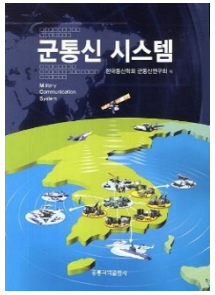
Homepage: http://mmcn.ajou.ac.kr

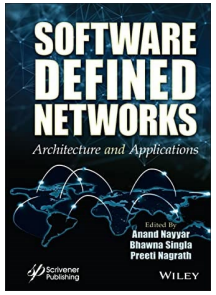
facebook: http://www.facebook.com/#!/groups/190702010947314

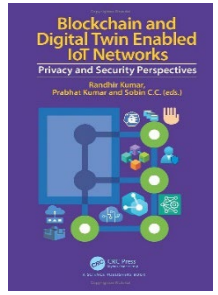
BK21 FOUR

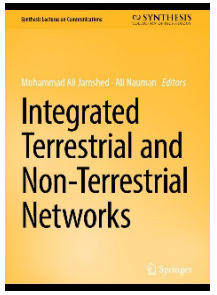
AI융합네트워크학과

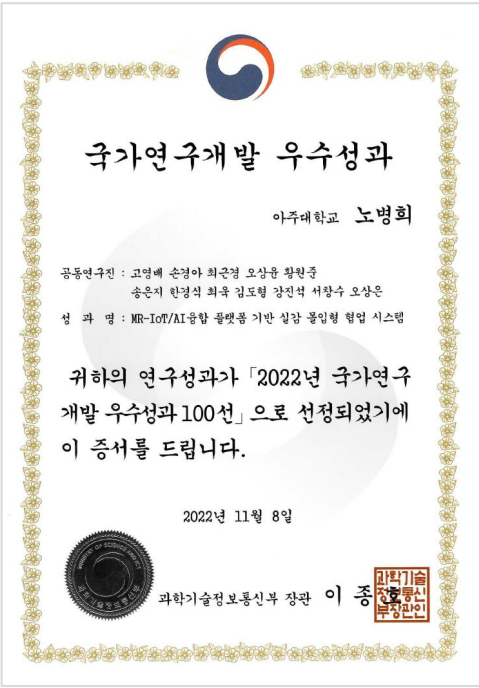












국가연구개발 우수성과

아주대학교 노병희

공동연구진 : 고영배 손경아 최준혁 오상윤 황원준
송은지 한경식 최욱 김도형 강진석 서창수 오상은

성 과 명 : MR-IoT/AI융합 플랫폼 기반 실감 몰입형 협업 시스템

귀하의 연구성과가 「2022년 국가연구개발 우수성과100선」으로 선정되었기에 이 증서를 드립니다.

2022년 11월 8일

과학기술정보통신부 장관 이 종 호